Universidad Politécnica de Aguascalientes
Sistemas Embebidos

CÓDIGO PROYECTO FINAL ETAPA 2

Maestra: Melanie Nicole Mata Barbosa
Grupo: ISC09B
Fecha: 27 de octubre de 2026

Estudiantes:
Iker Uriel Ramírez López – UP230186
Gustavo Figueroa Enríquez – UP230107
Mariel Isabel Cervantes Medina – UP230584
Francisco Rodríguez González – UP230115
Jonathan Paredes de Santos – UP230651

1. Descripción del sistema

Cruce vial de dos carriles con paso peatonal. Cada carril tiene un detector infrarrojo de llegada y uno de paso. El sistema opera en dos modos que se alternan con un botón; un LED verde de modo se enciende solo cuando está activo el modo ADAPTATIVO (apagado = modo FIJO):

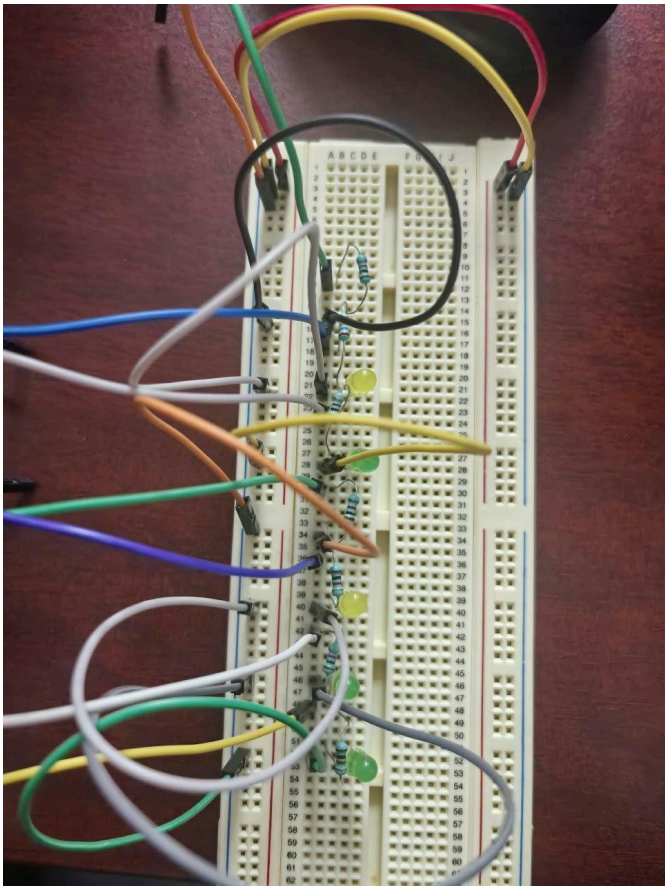
- **FIJO:** el verde dura siempre 10 s.
- **ADAPTATIVO:** verde = 5 s + 2 s × autos en cola + 1 s × llegadas en los últimos 30 s, limitado entre 5 y 20 s.

Fases: verde (según el modo), amarillo (2 s) y fase peatonal (6 s, ambos en rojo con buzzer intermitente cada 0.3 s). Un peatón se atiende después de 5 s mínimos de verde. Cada evento (llegada, paso con tiempo de espera, verde, solicitud peatonal, cambio de modo) se imprime y se guarda en telemetria_pendiente.jsonl para enviarse después a Azure.

2. Pines (Raspberry Pi 4, numeración BCM)

Señal	GPIO	Tipo
IR llegada carril 1 / carril 2	17 / 22	Entrada (pull-up)
IR paso carril 1 / carril 2	27 / 23	Entrada (pull-up)
Botón peatonal	24	Entrada (pull-up)
Botón de modo	20	Entrada (pull-up)
Semáforo 1: rojo / amarillo / verde	5 / 6 / 13	Salida
Semáforo 2: rojo / amarillo / verde	19 / 26 / 16	Salida
LED de modo (verde): encendido solo en modo ADAPTATIVO	25	Salida
Buzzer	4	Salida

3. Avance del montaje físico



Estado actual: el circuito está parcialmente armado en protoboard (LEDs, resistencias y cables de conexión). Faltan algunos materiales por conseguir. Los LED rojos de los semáforos están contemplados en el diseño pero todavía no se colocan. El LED de modo será únicamente verde (encendido en modo ADAPTATIVO).

4. Diagramas de flujo

Diagrama 1: flujo general del programa.

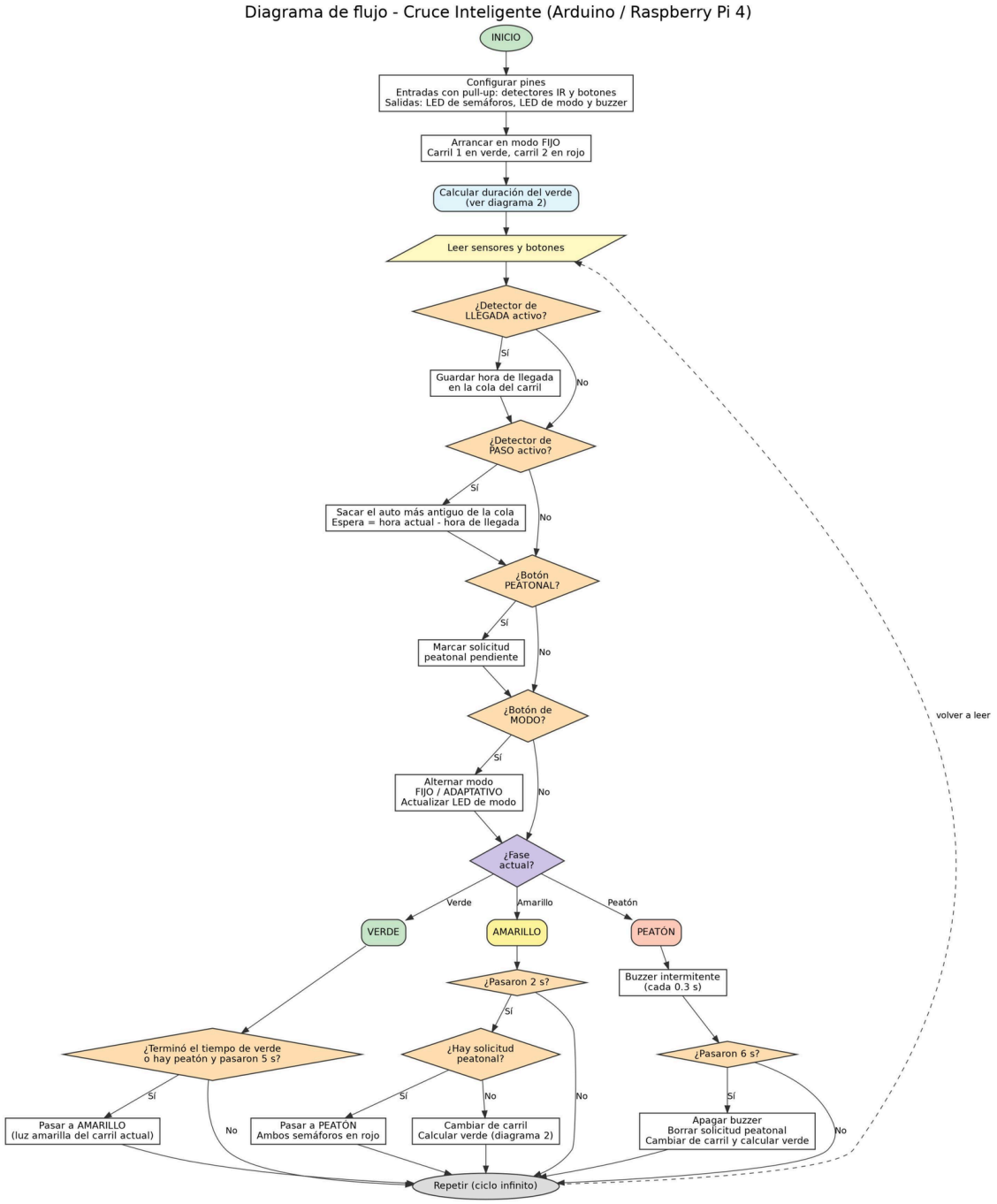
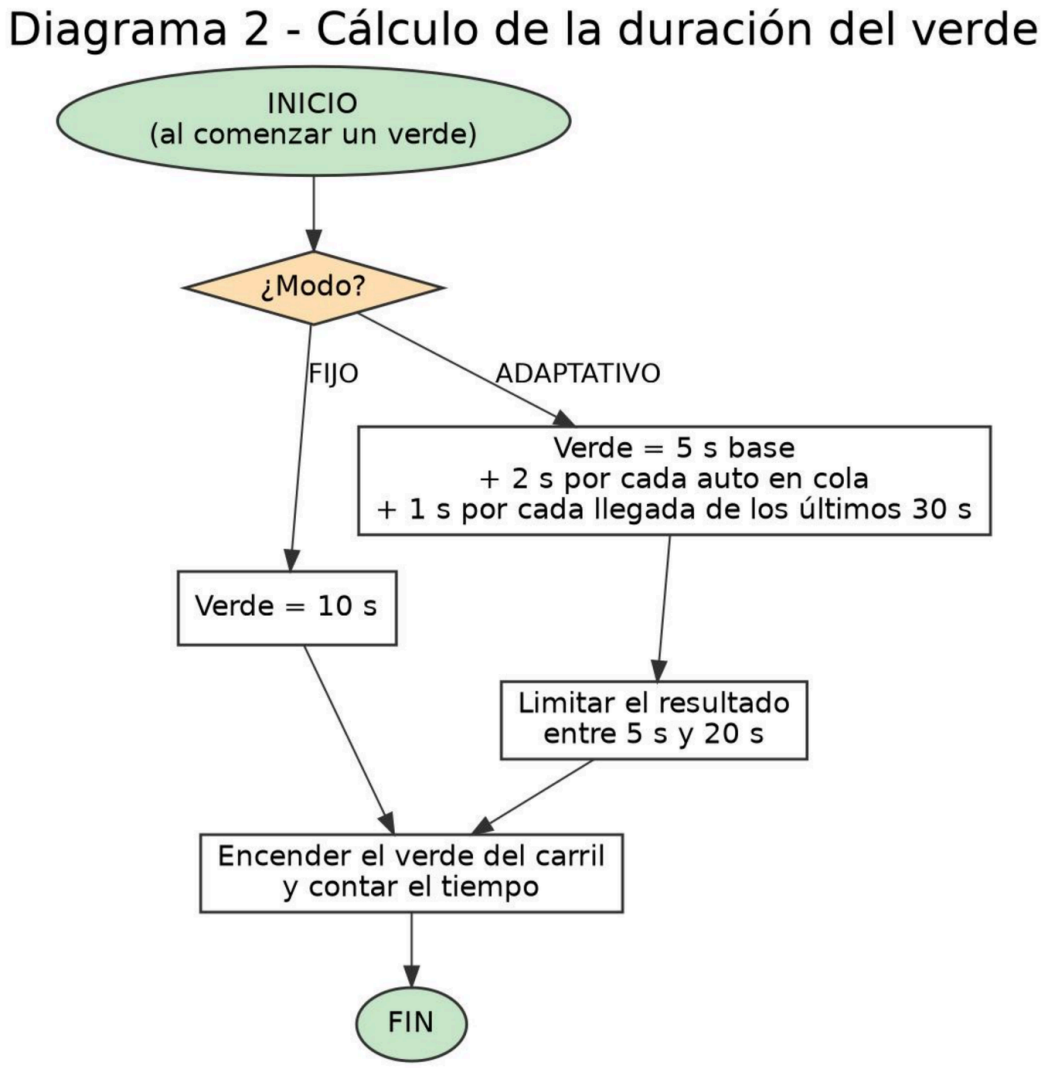


Diagrama 2: cálculo del verde en modo FIJO y ADAPTATIVO.



5. Código Arduino

Pines utilizados:

Señal	Pin Arduino	Tipo
IR llegada carril 1 / carril 2	2 / 4	Entrada (pull-up)
IR paso carril 1 / carril 2	3 / 5	Entrada (pull-up)
Botón peatonal	6	Entrada (pull-up)
Botón de modo	A4	Entrada (pull-up)
Semáforo 1: rojo / amarillo / verde	7 / 8 / 9	Salida
Semáforo 2: rojo / amarillo / verde	10 / 11 / 12	Salida
LED de modo: rojo / verde	A1 / A2	Salida
Buzzer	A3	Salida

```
/*

  Pines (ATmega328P -> numero Arduino):
  IR_C1_LLEGADA PD2=2   IR_C1_PASO PD3=3
  IR_C2_LLEGADA PD4=4   IR_C2_PASO PD5=5
  BTN_PEATON    PD6=6   (sin semaforo peatonal; avisa el buzzer)
  Semaforo 1: R PD7=7  A PB0=8   V PB1=9
  Semaforo 2: R PB2=10 A PB3=11  V PB4=12
  Modo:       R PC1=A1 V PC2=A2
  Buzzer      PC3=A3
  BTN_MODO    PC4=A4
  Todas las entradas son botones a tierra (activas en LOW).
*/

const uint8_t IR_LLEGADA[2] = {2, 4};
const uint8_t IR_PASO[2]    = {3, 5};
const uint8_t BTN_PEATON    = 6;
const uint8_t BTN_MODO      = A4;
const uint8_t SEM[2][3]     = {{7, 8, 9}, {10, 11, 12}}; // rojo, amarillo, verde
const uint8_t MODO_R = A1, MODO_V = A2;
const uint8_t BUZZER = A3;

// Tiempos (ms)
const unsigned long VERDE_FIJO   = 10000;
const unsigned long VERDE_MIN    = 5000;
const unsigned long VERDE_MAX    = 20000;
const unsigned long AMARILLO_T   = 2000;
const unsigned long PEATON_T     = 6000;
const unsigned long PEATON_MIN_V = 5000;   // verde minimo antes de atender peaton
const unsigned long VENTANA_TEND = 30000;  // ventana de tendencia
const unsigned long ANTIREBOTE   = 200;

enum Fase { VERDE, AMARILLO, PEATON };
enum Modo { FIJO, ADAPTATIVO };

Fase fase = VERDE;
Modo modo = FIJO;
uint8_t carril = 0;              // carril con verde
unsigned long faseInicio = 0;
unsigned long duracionVerde = VERDE_FIJO;
bool pedPendiente = false;

// Cola FIFO de llegadas por carril (para medir espera y tamano de cola)
const uint8_t N = 16;
unsigned long fifo[2][N];
uint8_t fHead[2] = {0, 0}, fCount[2] = {0, 0};
// Historial de llegadas (tendencia)
unsigned long hist[2][N];
uint8_t hIdx[2] = {0, 0};
unsigned long totalLlegadas[2] = {0, 0}, totalPasos[2] = {0, 0};

// Estado de entradas (flanco de bajada con antirrebote)
```

```
bool prevIR_L[2] = {HIGH, HIGH}, prevIR_P[2] = {HIGH, HIGH};
bool prevPea = HIGH, prevModo = HIGH;
unsigned long lastIR_L[2] = {0, 0}, lastIR_P[2] = {0, 0};
unsigned long lastPea = 0, lastModo = 0;

unsigned long buzzerT = 0;
bool buzzerOn = false;

bool flanco(uint8_t pin, bool &prev, unsigned long &last, unsigned long now) {
  bool v = digitalRead(pin);
  bool f = (prev == HIGH && v == LOW && now - last > ANTIREBOTE);
  if (f) last = now;
  prev = v;
  return f;
}

uint8_t cola(uint8_t c) { return fCount[c]; }

uint8_t llegadasRecientes(uint8_t c, unsigned long now) {
  uint8_t n = 0;
  for (uint8_t i = 0; i < N; i++)
    if (hist[c][i] > 0 && now - hist[c][i] <= VENTANA_TEND) n++;
  return n;
}

void registrarLlegada(uint8_t c, unsigned long now) {
  totalLlegadas[c]++;
  if (fCount[c] < N) {
    fifo[c][(fHead[c] + fCount[c]) % N] = now;
    fCount[c]++;
  }
  hist[c][hIdx[c]] = now;
  hIdx[c] = (hIdx[c] + 1) % N;
  Serial.print(F("Llegada carril ")); Serial.println(c + 1);
}

void registrarPaso(uint8_t c, unsigned long now) {
  if (fCount[c] == 0) return;
  unsigned long espera = now - fifo[c][fHead[c]];
  fHead[c] = (fHead[c] + 1) % N;
  fCount[c]--;
  totalPasos[c]++;
  Serial.print(F("Paso carril ")); Serial.print(c + 1);
  Serial.print(F(" | espera ms: ")); Serial.println(espera);
}

unsigned long calcularVerde(unsigned long now) {
  if (modo == FIJO) return VERDE_FIJO;
  unsigned long t = VERDE_MIN + 2000UL * cola(carril) + 1000UL * llegadasRecientes(carril, now);
  return constrain(t, VERDE_MIN, VERDE_MAX);
}

void apagarSemaforos() {
  for (uint8_t s = 0; s < 2; s++)
    for (uint8_t l = 0; l < 3; l++) digitalWrite(SEM[s][l], LOW);
}

void aplicarLuces() {
  apagarSemaforos();
  digitalWrite(MODO_R, modo == FIJO);
  digitalWrite(MODO_V, modo == ADAPTATIVO);
  if (fase == PEATON) {
    digitalWrite(SEM[0][0], HIGH); digitalWrite(SEM[1][0], HIGH);
    return;
  }
  uint8_t otro = 1 - carril;
  digitalWrite(SEM[otro][0], HIGH);              // el otro en rojo
  if (fase == VERDE) digitalWrite(SEM[carril][2], HIGH);
  else               digitalWrite(SEM[carril][1], HIGH);
}

void iniciarVerde(unsigned long now) {
  fase = VERDE;
  faseInicio = now;
  duracionVerde = calcularVerde(now);
  Serial.print(F("VERDE carril ")); Serial.print(carril + 1);
  Serial.print(F(" | ")); Serial.print(modo == FIJO ? F("fijo") : F("adaptativo"));
  Serial.print(F(" | cola ")); Serial.print(cola(carril));
  Serial.print(F(" | duracion s: ")); Serial.println(duracionVerde / 1000);
  aplicarLuces();
}
```

```
void setup() {
  for (uint8_t c = 0; c < 2; c++) {
    pinMode(IR_LLEGADA[c], INPUT_PULLUP);
    pinMode(IR_PASO[c], INPUT_PULLUP);
    for (uint8_t l = 0; l < 3; l++) pinMode(SEM[c][l], OUTPUT);
  }
  pinMode(BTN_PEATON, INPUT_PULLUP);
  pinMode(BTN_MODO, INPUT_PULLUP);
  pinMode(MODO_R, OUTPUT); pinMode(MODO_V, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  Serial.begin(9600);
  Serial.println(F("Cruce inteligente listo (modo fijo)"));
  iniciarVerde(millis());
}

void loop() {
  unsigned long now = millis();

  // --- Lectura de sensores ---
  for (uint8_t c = 0; c < 2; c++) {
    if (flanco(IR_LLEGADA[c], prevIR_L[c], lastIR_L[c], now)) registrarLlegada(c, now);
    if (flanco(IR_PASO[c],    prevIR_P[c], lastIR_P[c], now)) registrarPaso(c, now);
  }
  if (flanco(BTN_PEATON, prevPea, lastPea, now)) {
    pedPendiente = true;
    Serial.println(F("Peaton solicita cruce"));
  }
  if (flanco(BTN_MODO, prevModo, lastModo, now)) {
    modo = (modo == FIJO) ? ADAPTATIVO : FIJO;
    Serial.println(modo == FIJO ? F("Modo: FIJO") : F("Modo: ADAPTATIVO"));
    aplicarLuces();
  }

  // --- Maquina de estados del semaforo ---
  unsigned long t = now - faseInicio;
  switch (fase) {
    case VERDE:
      if (t >= duracionVerde || (pedPendiente && t >= PEATON_MIN_V)) {
        fase = AMARILLO; faseInicio = now; aplicarLuces();
      }
      break;
    case AMARILLO:
      if (t >= AMARILLO_T) {
        if (pedPendiente) {
          fase = PEATON; faseInicio = now; aplicarLuces();
          Serial.println(F("Fase peatonal"));
        } else {
          carril = 1 - carril;
          iniciarVerde(now);
        }
      }
      break;
    case PEATON:
      // buzzer intermitente
      if (now - buzzerT >= 300) { buzzerT = now; buzzerOn = !buzzerOn; digitalWrite(BUZZER, buzzerOn); }
      if (t >= PEATON_T) {
        digitalWrite(BUZZER, LOW); buzzerOn = false;
        pedPendiente = false;
        carril = 1 - carril;
        iniciarVerde(now);
      }
      break;
  }
}
```

6. Código Raspberry Pi (Python)

```
#!/usr/bin/env python3
"""
Cruce Inteligente - Raspberry Pi 4 (migracion del sketch de Arduino)
Modo FIJO y modo ADAPTATIVO, boton peatonal, boton de cambio de modo.

Pines BCM (numeracion GPIO de la Raspberry):
  IR_C1_LLEGADA GPIO17   IR_C1_PASO GPIO27
  IR_C2_LLEGADA GPIO22   IR_C2_PASO GPIO23
  BTN_PEATON    GPIO24
  BTN_MODO      GPIO20
  Semaforo 1: R GPIO5  A GPIO6   V GPIO13
  Semaforo 2: R GPIO19 A GPIO26  V GPIO16
  Modo:       R GPIO12 V GPIO25
  Buzzer      GPIO4
Entradas: modulo/boton a tierra, pull-up interna (activas en LOW).
Sensores y modulos alimentados a 3.3 V.

Uso en la Raspberry:  python3 cruce_inteligente_raspberry.py
Prueba en PC sin Raspberry:  SIMULAR=1 python3 cruce_inteligente_raspberry.py
"""
import json
import os
import random
import time
from collections import deque

SIMULAR = os.environ.get("SIMULAR") == "1"

if not SIMULAR:
    import RPi.GPIO as GPIO
else:
    class _GPIOFalso:
        BCM = OUT = IN = PUD_UP = 0
        HIGH, LOW = 1, 0
        def setmode(self, *a): pass
        def setwarnings(self, *a): pass
        def setup(self, *a, **k): pass
        def output(self, *a): pass
        def input(self, pin):
            # En simulacion: de vez en cuando "pasa" un vehiculo
            return 0 if random.random() < 0.03 else 1
        def cleanup(self): pass
    GPIO = _GPIOFalso()

# ---------------- Pines ----------------
IR_LLEGADA = [17, 22]
IR_PASO = [27, 23]
BTN_PEATON = 24
BTN_MODO = 20
SEM = [[5, 6, 13], [19, 26, 16]]   # rojo, amarillo, verde
MODO_R, MODO_V = 12, 25
BUZZER = 4

# ---------------- Tiempos (segundos) ----------------
VERDE_FIJO = 10
VERDE_MIN = 5
VERDE_MAX = 20
AMARILLO_T = 2
PEATON_T = 6
PEATON_MIN_V = 5          # verde minimo antes de atender al peaton
VENTANA_TEND = 30         # ventana de tendencia
ANTIREBOTE = 0.2
ARCHIVO_LOG = "telemetria_pendiente.jsonl"   # respaldo local (se enviara a Azure)

FIJO, ADAPTATIVO = "fijo", "adaptativo"
VERDE, AMARILLO, PEATON = "VERDE", "AMARILLO", "PEATON"


class Cruce:
    def __init__(self):
        self.modo = FIJO
        self.fase = VERDE
        self.carril = 0
        self.fase_inicio = 0.0
        self.duracion_verde = VERDE_FIJO
        self.ped_pendiente = False
```

```
        self.cola = [deque(), deque()]        # llegadas sin paso (marcas de tiempo)
        self.hist = [deque(), deque()]        # llegadas recientes (tendencia)
        self.total_llegadas = [0, 0]
        self.total_pasos = [0, 0]
        self.prev = {}                        # estado previo de cada entrada
        self.ultimo = {}                      # ultimo flanco (antirrebote)
        self.buzzer_t = 0.0
        self.buzzer_on = False

    # ---------- utilidades ----------
    def log(self, evento, **datos):
        registro = {"t": time.time(), "evento": evento, "modo": self.modo, **datos}
        print(registro)
        with open(ARCHIVO_LOG, "a") as f:
            f.write(json.dumps(registro) + "\n")

    def flanco(self, pin, ahora):
        """True cuando la entrada pasa de 1 a 0 (con antirrebote)."""
        v = GPIO.input(pin)
        ant = self.prev.get(pin, GPIO.HIGH)
        self.prev[pin] = v
        if ant == GPIO.HIGH and v == GPIO.LOW and ahora - self.ultimo.get(pin, 0) > ANTIREBOTE:
            self.ultimo[pin] = ahora
            return True
        return False

    def llegadas_recientes(self, c, ahora):
        while self.hist[c] and ahora - self.hist[c][0] > VENTANA_TEND:
            self.hist[c].popleft()
        return len(self.hist[c])

    # ---------- sensores ----------
    def registrar_llegada(self, c, ahora):
        self.total_llegadas[c] += 1
        self.cola[c].append(ahora)
        self.hist[c].append(ahora)
        self.log("llegada", carril=c + 1, cola=len(self.cola[c]))

    def registrar_paso(self, c, ahora):
        if not self.cola[c]:
            return
        espera = ahora - self.cola[c].popleft()
        self.total_pasos[c] += 1
        self.log("paso", carril=c + 1, espera_s=round(espera, 2))

    # ---------- semaforo ----------
    def calcular_verde(self, ahora):
        if self.modo == FIJO:
            return VERDE_FIJO
        t = VERDE_MIN + 2 * len(self.cola[self.carril]) + 1 * self.llegadas_recientes(self.carril, ahora)
        return max(VERDE_MIN, min(VERDE_MAX, t))

    def aplicar_luces(self):
        for s in SEM:
            for pin in s:
                GPIO.output(pin, GPIO.LOW)
        GPIO.output(MODO_R, self.modo == FIJO)
        GPIO.output(MODO_V, self.modo == ADAPTATIVO)
        if self.fase == PEATON:
            GPIO.output(SEM[0][0], GPIO.HIGH)
            GPIO.output(SEM[1][0], GPIO.HIGH)
            return
        otro = 1 - self.carril
        GPIO.output(SEM[otro][0], GPIO.HIGH)                      # el otro en rojo
        GPIO.output(SEM[self.carril][2 if self.fase == VERDE else 1], GPIO.HIGH)

    def iniciar_verde(self, ahora):
        self.fase = VERDE
        self.fase_inicio = ahora
        self.duracion_verde = self.calcular_verde(ahora)
        self.log("verde", carril=self.carril + 1,
                 cola=len(self.cola[self.carril]), duracion_s=self.duracion_verde)
        self.aplicar_luces()

    # ---------- ciclo principal ----------
    def paso_logica(self, ahora):
        # Lectura de sensores y botones
        for c in range(2):
            if self.flanco(IR_LLEGADA[c], ahora):
                self.registrar_llegada(c, ahora)
            if self.flanco(IR_PASO[c], ahora):
                self.registrar_paso(c, ahora)
```

```
        if self.flanco(BTN_PEATON, ahora):
            self.ped_pendiente = True
            self.log("peaton_solicita")
        if self.flanco(BTN_MODO, ahora):
            self.modo = ADAPTATIVO if self.modo == FIJO else FIJO
            self.log("cambio_modo")
            self.aplicar_luces()

        # Maquina de estados
        t = ahora - self.fase_inicio
        if self.fase == VERDE:
            if t >= self.duracion_verde or (self.ped_pendiente and t >= PEATON_MIN_V):
                self.fase, self.fase_inicio = AMARILLO, ahora
                self.aplicar_luces()
        elif self.fase == AMARILLO:
            if t >= AMARILLO_T:
                if self.ped_pendiente:
                    self.fase, self.fase_inicio = PEATON, ahora
                    self.aplicar_luces()
                    self.log("fase_peatonal")
                else:
                    self.carril = 1 - self.carril
                    self.iniciar_verde(ahora)
        elif self.fase == PEATON:
            if ahora - self.buzzer_t >= 0.3:          # buzzer intermitente
                self.buzzer_t = ahora
                self.buzzer_on = not self.buzzer_on
                GPIO.output(BUZZER, self.buzzer_on)
            if t >= PEATON_T:
                GPIO.output(BUZZER, GPIO.LOW)
                self.buzzer_on = False
                self.ped_pendiente = False
                self.carril = 1 - self.carril
                self.iniciar_verde(ahora)


def main():
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)
    for pin in IR_LLEGADA + IR_PASO + [BTN_PEATON, BTN_MODO]:
        GPIO.setup(pin, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    for s in SEM:
        for pin in s:
            GPIO.setup(pin, GPIO.OUT, initial=GPIO.LOW)
    for pin in (MODO_R, MODO_V, BUZZER):
        GPIO.setup(pin, GPIO.OUT, initial=GPIO.LOW)

    cruce = Cruce()
    print("Cruce inteligente listo (modo fijo). Ctrl+C para salir.")
    cruce.iniciar_verde(time.monotonic())
    try:
        while True:
            cruce.paso_logica(time.monotonic())
            time.sleep(0.01)                          # 10 ms por vuelta
    except KeyboardInterrupt:
        print("\nSaliendo...")
    finally:
        GPIO.cleanup()


if __name__ == "__main__":
    main()
```